⚖️ SOFTWARE DESIGN DOCUMENT

JusticeChain

Blockchain-Based Justice Management System

Field	Details
Document Type	Software Design Document (SDD)
Version	1.0
Date	March 11, 2026
Author	Janani K — 110822104031
Guide	Mrs. Rajalakshmi, Asst. Professor
Standard	IEEE Std 1016-2009
Status	☑ Final

TABLE OF CONTENTS

> **Navigation Guide** — Each section below corresponds to a major design view of the JusticeChain system. Sections progress from high-level architecture down to component-level specifications.


```
PART I   — INTRODUCTION & SCOPE ............... Section 1
PART II  — SYSTEM ARCHITECTURE ................ Section 2
PART III — COMPONENT DESIGN ................... Section 3
PART IV  — DATABASE DESIGN .................... Section 4
PART V   — SMART CONTRACT DESIGN .............. Section 5
PART VI  — INTERFACE DESIGN ................... Section 6
PART VII — SECURITY DESIGN .................... Section 7
PART VIII— DEPLOYMENT DESIGN .................. Section 8
APPENDIX — DIAGRAMS & REFERENCES .............. Section 9
```


PART I : INTRODUCTION & SCOPE

1.1 ◆ Document Purpose

This Software Design Document (SDD) defines the **architectural and component-level design** of the JusticeChain platform. It translates the requirements captured in the Software Requirements Specification

(SRS) into concrete design decisions, module structures, data models, and interface contracts that guide the development team during implementation.

> 📌 **Intended Audience**
>
> - Software Engineers & Full-Stack Developers
> - Blockchain Developers (Solidity / Web3)
> - Database Architects (MongoDB)
> - UI/UX Designers
> - QA / Testing Engineers
> - Project Guide & Evaluators

1.2 ◆ Design Goals & Principles

The following design principles govern every decision made in this document:

1. **Separation of Concerns** — Frontend, Backend, Blockchain, and Storage layers are independently deployable units
2. **Immutability First** — All critical justice data is anchored to the Ethereum blockchain
3. **Least Privilege Access** — Each role can only access the data and operations it is permitted to perform
4. **Decentralized Storage** — Evidence files are never stored on a centralized server; IPFS is used exclusively
5. **Tamper Evidence** — SHA-256 + blockchain hashing ensures any modification is immediately detectable
6. **Stateless API** — All REST endpoints are stateless; sessions are managed via JWT tokens
7. **Audit by Default** — Every user action generates an immutable activity log

---

1.3 ◆ Scope of Design

> ☑ **In Scope**
>
> - React 18 Single Page Application (Frontend)
> - Node.js / Express REST API (Backend)
> - MongoDB Atlas (Database)
> - Ethereum Smart Contract — `JusticeChain.sol` (Blockchain)
> - Pinata IPFS Gateway (Decentralized File Storage)
> - JWT + Role-Based Authentication (Security)
> - EmailJS / Nodemailer (Notification System)

> ❌ **Out of Scope**
>
> - Mobile applications (iOS / Android)
> - Physical evidence tracking hardware
> - Payment processing
> - External court scheduling systems

1.4 ◆ Definitions Used in This Document

Symbol	Meaning
[FE]	Belongs to Frontend Layer
[BE]	Belongs to Backend Layer
[BC]	Belongs to Blockchain Layer
[DB]	Belongs to Database Layer
[FS]	Belongs to File Storage Layer
◆	Key design decision point
⚡	Performance-sensitive component
🔒	Security-critical component

PART II : SYSTEM ARCHITECTURE

2.1 ◆ Architecture Style

JusticeChain adopts a **Layered + Microservice-Hybrid Architecture** composed of five distinct tiers:

```
╔═══════════════════════════════════════════════════════════════╗
║                    PRESENTATION TIER  [FE]                    ║
║         React 18 + Vite + Tailwind CSS + Web3Modal            ║
║    [ Home | Login | Register | 4× Role Dashboards ]           ║
╟═══════════════════════════════════════════════════════════════╢
║                    APPLICATION TIER  [BE]                     ║
║           Node.js v18 + Express.js REST API                   ║
║   [ Auth | Cases | Evidence | Admin | Reports Routes ]        ║
╟═══════════════════════════════════════════════════════════════╢
║                     DATA TIER  [DB]                           ║
║           MongoDB Atlas (Document Store)                      ║
║ [ User | Case | Evidence | Suspect | Witness | AuditLog ]     ║
╟═══════════════════════════════════════════════════════════════╢
║                  BLOCKCHAIN TIER  [BC]                        ║
║       Ethereum Testnet + JusticeChain.sol Smart Contract      ║
║  [ Role Registry | Case Anchor | Evidence Hash | Verdicts ]   ║
╟═══════════════════════════════════════════════════════════════╢
║                   STORAGE TIER  [FS]                          ║
║           Pinata IPFS Gateway                                 ║
║      [ Evidence Files | Forensic Reports | Documents ]        ║
╚═══════════════════════════════════════════════════════════════╝
```

2.2 ◆ High-Level Component Interaction

```
User's Browser ──────► ┌──────────────────┐
                       │   React SPA  [FE]  │
                       └──────────────────┘
                                │
                                │  HTTP/REST (JWT)
                       ┌────────▼─────────┐
                       │  Express API  [BE]  │◄──── EmailJS
                       └──────┬──────────┘
              MongoDB ◄───────┘          └──────────────► Pinata IPFS
                                                │
                                       ┌────────▼──────────┐
                                       │ Ethereum Network  │
                                       │  JusticeChain.sol │
                                       └──────────────────┘
                                       (via ethers.js / web3)
```

2.3 ◆ Technology Stack Decision Matrix

Layer	Technology	Version	Justification
Frontend Framework	React	18.2.0	Component-based UI, large ecosystem
Build Tool	Vite	5.0.0	Fast HMR, optimized builds
Styling	Tailwind CSS	3.3.0	Utility-first, no CSS bloat
Web3 Connection	Wagmi + Web3Modal	2.x / 5.x	Wallet abstraction across providers
Ethereum Library	ethers.js	6.9.0	Modern Web3 interaction
Backend Runtime	Node.js	18+	Async I/O, JS ecosystem
Backend Framework	Express.js	4.18.2	Minimal, flexible routing
Database	MongoDB + Mongoose	7.7.0	Flexible schema for complex case data
Authentication	JWT (jsonwebtoken)	9.0.2	Stateless, scalable auth
Password Security	bcryptjs	2.4.3	Salted hash for password storage
File Upload	Multer	2.0.2	Multipart form handling
IPFS Pinning	Pinata SDK	Latest	Reliable IPFS pinning service
Blockchain	Solidity	0.8.19	Type-safe smart contract language
Email	Nodemailer / EmailJS	Latest	Role ID delivery on registration
Deployment (FE)	Vercel	—	Serverless static hosting
Deployment (BE)	Render	—	Persistent Node.js hosting

2.4 ◆ Request Lifecycle

> Every user request in JusticeChain follows this standardized lifecycle:

```
Step 1 ➔ User triggers action in React UI
Step 2 ➔ React sends HTTP request with JWT Bearer token
Step 3 ➔ Express authMiddleware validates JWT
Step 4 ➔ Role Authorization check (RBAC)
Step 5 ➔ Controller processes business logic
Step 6 ➔ MongoDB query / mutation executed
Step 7 ➔ (If blockchain action) blockchainService.js called
Step 8 ➔ (If file action) pinataService.js called
Step 9 ➔ Response returned to frontend
Step 10 ► React state updated → UI re-renders
```

PART III : COMPONENT DESIGN

3.1 ◆ Frontend Component Hierarchy

```
App.jsx
├── WagmiProvider + QueryClientProvider
│   └── Router (React Router v6)
│       ├── Layout.jsx  [Public Routes]
│       │   ├── Home.jsx
│       │   ├── About.jsx
│       │   ├── Services.jsx
│       │   └── Contact.jsx
│       ├── Login.jsx             [Auth Route]
│       ├── Register.jsx          [Auth Route]
│       └── ProtectedRoute.jsx
│           ├── AdminDashboard.jsx     [ADMIN only]
│           ├── PoliceDashboard.jsx    [POLICE only]
│           ├── ForensicDashboard.jsx  [FORENSIC only]
│           └── JudgeDashboard.jsx     [JUDGE only]
│
└── Shared Components
    ├── DashboardSwitcher.jsx
    ├── PasswordStrength.jsx
    ├── RoleSelect.jsx
    └── WalletConnect.jsx
```

3.2 ◆ Component Specifications

► `ProtectedRoute.jsx` 🔒

> **Purpose:** Guards role-specific routes from unauthorized access.

```
Inputs  : { children, allowedRole }
Logic   : Read session from localStorage → compare role →
          → Match    : render children
          → No match : redirect to /login
          → Suspended: redirect to / with alert
Outputs : JSX Children or <Navigate />
```

► `Login.jsx`

> **Purpose:** Authenticates users and initiates a session.

```
Form Fields   : email (string), password (string)
On Submit     : POST /api/auth/login
Success Flow  : Store JWT + user object in localStorage
               → Redirect to /dashboard/{role}
Error Handling: Toast notification for invalid credentials
State         : { email, password, loading, error }
```

► `Register.jsx`

> **Purpose:** Creates new user accounts with role selection.

```
Form Fields   : username, email, password, confirmPassword, role
               (ADMIN | POLICE | FORENSIC | JUDGE)
On Submit     : POST /api/auth/register
Success Flow  : Role ID sent to email → Redirect to /login
Components    : <PasswordStrength /> for real-time feedback
               <RoleSelect /> dropdown
State         : { formData, passwordStrength, loading }
```

► `AdminDashboard.jsx` 🔒

> **Purpose:** Central control panel for system administrators.

```
Tabs          : Dashboard | Cases | Users | Evidence |
                Blockchain | Audit Logs | System
Data Sources  : GET /api/cases/all
                GET /api/admin/users
                GET /api/admin/audit-logs
                GET /api/admin/health
```

```
Refresh        : Auto every 30 seconds (setInterval)
Key Actions    : approveCase(), assignForensic(),
                 assignJudge(), suspendUser()
State Vars     : 11 (cases, users, stats, activeTab,
                 loading, error, auditLogs,
                 blockchainStatus, systemHealth ...)
```

► `PoliceDashboard.jsx`

> **Purpose:** Case management interface for police officers.

```
Key Features   : Create Case → Upload Evidence →
                 Track Status → Add Investigation Notes →
                 Add Suspects → Add Witnesses
API Calls      : POST /api/cases/create
                 GET  /api/cases/my-cases
                 POST /api/evidence/upload
                 POST /api/:caseId/suspects
                 POST /api/:caseId/witnesses
Blockchain     : createCase() called after DB insert
                addEvidence() called after IPFS upload
```

► `ForensicDashboard.jsx`

> **Purpose:** Evidence analysis workspace for forensic officers.

```
Key Features   : View assigned cases → Analyze evidence →
                 Submit forensic reports → Update evidence status
API Calls      : GET /api/cases/my-cases
                 GET /api/evidence/:caseId
                 PUT /api/evidence/:id/analyze
Blockchain     : submitForensicReport() after analysis
IPFS           : Analysis report pinned to IPFS
                (Pinata), hash stored on chain
```

► `JudgeDashboard.jsx`

> **Purpose:** Judicial review and verdict management.

```
Key Features   : Review assigned cases → Review forensic reports →
                 Schedule hearings → Submit verdicts
API Calls      : GET  /api/cases/my-cases
                 POST /api/cases/:caseId/hearings
```

```
                 PUT  /api/cases/:caseId/verdict
Blockchain     : giveVerdict() → closes case on chain
                (case.closed = true, immutable)
```

► `WalletConnect.jsx`

> **Purpose:** Web3 wallet integration for blockchain identity.

```
Library        : Wagmi v2 + Web3Modal v5
Networks       : Ethereum Mainnet / Sepolia Testnet
Function       : Connect MetaMask / WalletConnect
                 Store wallet address in user profile
                 Used for smart contract signing
```

3.3 ◆ Backend Module Structure

```
backend/
├── server.js           ← Entry point (port binding, DB connect)
├── app.js              ← Express config (CORS, middleware, routes)
├── config/
│   └── env.js          ← Environment variable loader
├── middleware/
│   └── authMiddleware.js   ← JWT verify + Role authorization
├── routes/             ← Route → Controller binding
│   ├── authRoutes.js
│   ├── caseRoutes.js
│   ├── evidenceRoutes.js
│   ├── adminRoutes.js
│   └── reportRoutes.js
├── models/             ← Mongoose schemas
│   ├── User.js
│   ├── Case.js
│   ├── Evidence.js
│   ├── Suspect.js
│   ├── Witness.js
│   ├── InvestigationNote.js
│   ├── Report.js
│   └── ActivityLog.js
└── utils/
    ├── blockchainService.js  ← ethers.js → smart contract calls
    ├── pinataService.js      ← IPFS file pin/unpin
    └── emailService.js       ← Nodemailer / EmailJS
```

3.4 ◆ API Endpoint Reference

Authentication Endpoints

Method	Endpoint	Auth	Role	Description
POST	`/api/auth/register`	❌	Any	Register new user
POST	`/api/auth/login`	❌	Any	Login & get JWT
POST	`/api/auth/logout`	☑	Any	Invalidate session
GET	`/api/auth/user/me`	☑	Any	Get current user
GET	`/api/auth/users`	☑	ADMIN	Get all users
GET	`/api/auth/active-sessions`	☑	ADMIN	Active sessions
POST	`/api/auth/check-verification`	❌	Any	Check email verified

Case Management Endpoints

Method	Endpoint	Auth	Role	Description
POST	`/api/cases/create`	☑	POLICE	Create new case
GET	`/api/cases/all`	☑	ADMIN	Get all cases
GET	`/api/cases/my-cases`	☑	All	Role-filtered cases
GET	`/api/cases/:caseId`	☑	All	Get case by ID
PUT	`/api/cases/:caseId/approve`	☑	ADMIN	Approve case
PUT	`/api/cases/:caseId/status`	☑	All	Update status
PUT	`/api/cases/:caseId/verdict`	☑	JUDGE	Submit verdict
PUT	`/api/cases/:caseId/blockchain`	☑	BE	Store blockchain info
POST	`/api/cases/assign-forensic`	☑	ADMIN	Assign forensic officer
POST	`/api/cases/assign-judge`	☑	ADMIN	Assign judge
POST	`/api/cases/:caseId/hearings`	☑	JUDGE	Schedule hearing
GET	`/api/cases/:caseId/hearings`	☑	All	Get hearing list

Investigation Endpoints

Method	Endpoint	Auth	Role	Description
POST	`/api/cases/:id/investigation-notes`	☑	POLICE/FORENSIC	Add note
GET	`/api/cases/:id/investigation-notes`	☑	All	Get notes
POST	`/api/cases/:id/suspects`	☑	POLICE	Add suspect
GET	`/api/cases/:id/suspects`	☑	All	Get suspects

Method	Endpoint	Auth	Role	Description
POST	`/api/cases/:id/witnesses`	☑	POLICE	Add witness
GET	`/api/cases/:id/witnesses`	☑	All	Get witnesses
GET	`/api/cases/:id/activity-logs`	☑	ADMIN	Audit logs

Evidence Endpoints

Method	Endpoint	Auth	Role	Description
POST	`/api/evidence/upload`	☑	POLICE	Upload evidence file
GET	`/api/evidence/:caseId`	☑	All	Get case evidence
PUT	`/api/evidence/:id/analyze`	☑	FORENSIC	Submit analysis
PUT	`/api/evidence/:id/verify`	☑	FORENSIC	Verify integrity
PUT	`/api/evidence/:id/lock`	☑	ADMIN	Lock evidence

PART IV : DATABASE DESIGN

4.1 ◆ Database Technology Choice

> JusticeChain uses **MongoDB Atlas** (Cloud-hosted NoSQL) as its primary data store. The document-oriented model is ideal for nested case data (timeline events, hearing history, chain of custody) that would require multiple JOIN tables in a relational database.

4.2 ◆ Entity Relationship Overview

```
User ──────────────────── registers ──► Case
  │                                      │
  │ (assignedForensic)                   ├── Evidence[]
  │ (assignedJudge)                      ├── Suspect[]
  │ (registeredBy)                       ├── Witness[]
  └──────────────────────────────────────├── InvestigationNote[]
                                         ├── Timeline[]
                                         ├── HearingHistory[]
                                         └── ActivityLog[]

Evidence ─────────────── has ──────────► ChainOfCustody[]
```

4.3 ◆ Collection Schemas

Collection: `users`

```
Field           Type        Constraints              Description
──────────────────────────────────────────────────────────────────
_id             ObjectId    PK, auto                 MongoDB ID
username        String      required, unique, trim   Login name
email           String      required, lowercase      Contact email
password        String      required, bcrypt hashed  Auth secret
role            String      enum: ADMIN|POLICE|      User role
                            FORENSIC|JUDGE
roleId          String      required, unique         System role ID
wallet          String      nullable                 ETH wallet addr
isVerified      Boolean     default: false           Email verified
isSuspended     Boolean     default: false           Account state
createdAt       Date        default: now             Registration date
```

Collection: `cases`

```
Field                   Type        Description
──────────────────────────────────────────────────────────────
_id                     ObjectId    MongoDB ID
caseId                  String      Unique case identifier
caseNumber              String      Official case number
title                   String      Case title
description             String      Detailed description
status                  String      DRAFT|REGISTERED|
                                    PENDING_APPROVAL|APPROVED|
                                    IN_FORENSIC_ANALYSIS|
                                    ANALYSIS_COMPLETE|
                                    HEARING|CLOSED
priority                String      LOW|MEDIUM|HIGH|CRITICAL
isDraft                 Boolean     Draft save state
registeredBy            ObjectId    Ref → User (Police)
policeStation           String      Station name
assignedForensic        ObjectId    Ref → User (Forensic)
assignedJudge           ObjectId    Ref → User (Judge)
approvedBy              ObjectId    Ref → User (Admin)
evidence                ObjectId[]  Ref → Evidence[]
suspects                ObjectId[]  Ref → Suspect[]
witnesses               ObjectId[]  Ref → Witness[]
timeline                Embedded[]  Status change history
transferRequest         Embedded    Transfer to other station
hearingHistory          Embedded[]  Scheduled hearings
verdictDecision         String      GUILTY|NOT_GUILTY|etc.
verdictSummary          String      Verdict text
blockchainCaseId        Number      On-chain case ID
blockchainHash          String      TX hash
blockchainCaseTxHash    String      Case creation TX
blockchainApprovalTxHash String     Approval TX
```

```
smartContractAddress    String      Contract address
location                String      Incident location
latitude / longitude    Number      GPS coordinates
createdAt / updatedAt   Date        Timestamps
closedAt                Date        Case closure time
```

Collection: `evidences`

```
Field                Type        Description
──────────────────────────────────────────────────────────
evidenceId           String      Unique identifier
caseId               ObjectId    Ref → Case
type                 String      DOCUMENT|IMAGE|VIDEO|
                                 AUDIO|DIGITAL|PHYSICAL
title                String      Evidence title
description          String      Description
uploadedBy           ObjectId    Ref → User
fileName             String      Original filename
fileSize             Number      Bytes
mimeType             String      MIME type
ipfsHash             String      IPFS content hash (CID)
pinataUrl            String      Pinata gateway URL
blockchainHash       String      On-chain evidence hash
sha256Hash           String      File integrity hash (SHA-256)
blockchainTxHash     String      Blockchain TX hash
status               String      UPLOADED|PENDING_ANALYSIS|
                                 ANALYZING|ANALYSIS_COMPLETE|
                                 VERIFIED|IMMUTABLE
isLocked             Boolean     Admin lock status
isVerified           Boolean     Forensic verification
analysisStatus       String      PENDING|IN_PROGRESS|
                                 COMPLETE|REJECTED
analyzedBy           ObjectId    Ref → User (Forensic)
analysisReport       String      Text report
analysisIpfsHash     String      IPFS hash of report
chainOfCustody       Embedded[]  Access + action history
geoLocation          Embedded    lat/lon/address
tamperDetectedAt     Date        Tamper detection timestamp
uploadedAt           Date        Upload timestamp
```

Sub-document: `chainOfCustody` **(within Evidence)**

```
Field            Type        Description
──────────────────────────────────────────────────
action           String      UPLOADED|ACCESSED|VERIFIED|
                             LOCKED|TRANSFERRED|
```

```
                              TAMPER_DETECTED
performedBy     ObjectId      Ref → User
performedByRole String        Role at time of action
timestamp       Date          Action time
details         String        Notes
hash            String        Hash snapshot at that moment
```

Collection: `suspects`

```
name, age, gender, address, nationalId,
description, status (ACTIVE|CLEARED|ARRESTED|CONVICTED),
addedBy (ObjectId → User), addedAt (Date)
```

Collection: `witnesses`

```
name, contactInfo, address,
statement, reliabilityScore (0–10),
addedBy (ObjectId → User), addedAt (Date)
```

Collection: `investigationnotes`

```
caseId (ObjectId), content (String), addedBy (ObjectId),
type (OBSERVATION|LEAD|FINDING|UPDATE), addedAt (Date)
```

Collection: `activitylogs`

```
caseId (ObjectId), performedBy (ObjectId → User),
action (String), details (String),
ipAddress (String), timestamp (Date)
```

Collection: `usersessions`

```
userId (ObjectId), token (String, hashed),
loginAt (Date), lastActive (Date),
ipAddress (String), userAgent (String),
isActive (Boolean)
```

4.4 ◆ Indexing Strategy ⚡

> The following fields are indexed to ensure query performance with large datasets:

```
Collection      Field           Index Type      Reason
──────────────────────────────────────────────────────────────────────
users           email           Unique          Login lookup
users           roleId          Unique          Role verification
cases           caseId          Unique          Case lookup by ID
cases           status          Single          Filter by status
cases           registeredBy    Single          Police case list
cases           assignedForensic Single         Forensic case list
cases           assignedJudge   Single          Judge case list
evidences       evidenceId      Unique          Evidence lookup
evidences       caseId          Single          Case evidence join
evidences       ipfsHash        Single          IPFS verification
activitylogs    caseId          Single          Audit log lookup
activitylogs    timestamp       Single          Date-range queries
```

PART V : SMART CONTRACT DESIGN

5.1 ◆ Contract Overview

> **Contract Name:** `JusticeChain.sol` **Language:** Solidity ^0.8.19 **License:** MIT **Network:** Ethereum (Sepolia Testnet / Mainnet) **Deployment:** Admin wallet becomes contract owner on deploy

---

5.2 ◆ Contract Architecture

```
JusticeChain Contract
│
├── STATE VARIABLES
│   ├── address public admin
│   ├── address public backend
│   ├── mapping(address => bool) police
│   ├── mapping(address => bool) forensic
│   ├── mapping(address => bool) judge
│   ├── mapping(uint => CaseFile) cases
│   ├── mapping(uint => Evidence[]) evidences
│   ├── mapping(uint => ForensicReport) reports
│   ├── mapping(uint => Verdict) verdicts
│   ├── mapping(uint => AccessLog[]) accessLogs
```

```
│   └── uint totalCases
│
├── STRUCTS
│   ├── CaseFile      { caseId, policeOfficer, forensicOfficer,
│   │                   judgeOfficer, approved, closed }
│   ├── Evidence      { ipfsHash, hash (bytes32), uploadedBy, timestamp }
│   ├── ForensicReport{ ipfsHash, hash (bytes32), forensicOfficer, timestamp }
│   ├── Verdict       { decision, judgeOfficer, timestamp }
│   └── AccessLog     { user, timestamp }
│
├── MODIFIERS
│   ├── onlyAdmin
│   ├── onlyAdminOrBackend
│   ├── onlyPolice
│   ├── onlyForensic
│   └── onlyJudge
│
└── FUNCTIONS (grouped by role)
    ├── ADMIN    → registerPolice/Forensic/Judge, approveCase, assignCase
    ├── POLICE   → createCase, addEvidence
    ├── FORENSIC → submitForensicReport
    ├── JUDGE    → giveVerdict
    └── PUBLIC   → logAccess, verifyEvidence, verifyReport, hasAnyRole
```

5.3 ◆ Function Specifications

Admin Functions

```
registerPolice(address _addr)
  Modifier  : onlyAdminOrBackend
  Effect    : police[_addr] = true
  Event     : RoleRegistered(_addr, "POLICE")

approveCase(uint caseId)
  Modifier  : onlyAdmin
  Effect    : cases[caseId].approved = true
  Event     : CaseApproved(caseId)

assignCase(uint caseId, address forensicAddr, address judgeAddr)
  Modifier  : onlyAdmin
  Validates : forensic[forensicAddr] == true
              judge[judgeAddr] == true
  Effect    : Sets forensicOfficer & judgeOfficer in CaseFile
```

Police Functions

```
createCase()
  Modifier  : onlyPolice
  Effect    : totalCases++
              cases[totalCases] = new CaseFile(...)
  Event     : CaseCreated(totalCases, msg.sender)

addEvidence(uint caseId, string memory ipfsHash)
  Modifier  : onlyPolice
  Requires  : cases[caseId].approved == true
              cases[caseId].closed == false
  Effect    : hash = keccak256(ipfsHash)
              evidences[caseId].push(Evidence(...))
  Event     : EvidenceAdded(caseId, ipfsHash)
```

Forensic Functions

```
submitForensicReport(uint caseId, string memory ipfsHash)
  Modifier  : onlyForensic
  Requires  : cases[caseId].forensicOfficer == msg.sender
              cases[caseId].closed == false
  Effect    : hash = keccak256(ipfsHash)
              reports[caseId] = ForensicReport(...)
  Event     : ForensicSubmitted(caseId, ipfsHash)
```

Judge Functions

```
giveVerdict(uint caseId, string memory decision)
  Modifier  : onlyJudge
  Requires  : cases[caseId].judgeOfficer == msg.sender
              reports[caseId].timestamp > 0 (forensic done)
  Effect    : verdicts[caseId] = Verdict(decision, ...)
              cases[caseId].closed = true  ← IRREVERSIBLE
  Event     : VerdictGiven(caseId, decision)
```

5.4 ◆ Emitted Events (Blockchain Audit Trail)

```
Event                   Indexed Fields      When Emitted
──────────────────────────────────────────────────────────────
RoleRegistered          wallet, role        Role assigned
BackendAddressUpdated   newBackend          Backend wallet changed
CaseCreated             caseId, police      New case on chain
CaseApproved            caseId              Admin approves case
```

```
EvidenceAdded           caseId, ipfsHash    Evidence anchored
ForensicSubmitted       caseId, ipfsHash    Report anchored
VerdictGiven            caseId, decision    Case closed on chain
Accessed                caseId, user        View access logged
```

5.5 ◆ Backend ↔ Contract Integration (blockchainService.js)

```
Function                    Contract Call           Trigger Point
──────────────────────────────────────────────────────────────────────

registerUserRole()          registerPolice/         On user approval
                            Forensic/Judge()          by admin
createBlockchainCase()      createCase()            After DB case create
approveBlockchainCase()     approveCase()           After DB approval
addBlockchainEvidence()     addEvidence()           After IPFS upload
submitForensicReport()      submitForensicReport()  After analysis done
giveBlockchainVerdict()     giveVerdict()           After judge verdict
verifyEvidenceHash()        verifyEvidence()        Integrity check
```

PART VI : INTERFACE DESIGN

6.1 ◆ UI Design Principles

> The JusticeChain UI follows these design constraints:
>
> 1. **Dark-themed Dashboards** — Professional look fitting a justice system
> 2. **Color-coded Status Badges** — PENDING=yellow, APPROVED=green, CLOSED=grey, FORENSIC=purple
> 3. **Responsive Grid Layout** — Tailwind CSS grid adapts from mobile to desktop
> 4. **Toast Notifications** — react-toastify for success/error feedback
> 5. **Tab-based Navigation** — Dashboards use horizontal tab bars (no page reloads)
> 6. **Auto-refresh Panels** — Stats panels refresh every 30 seconds without user action

---

6.2 ◆ Page & Route Map

```
Route                   Component               Access
──────────────────────────────────────────────────────────────

/                       Home.jsx                Public
/about                  About.jsx               Public
/services               Services.jsx            Public
/contact                Contact.jsx             Public
/login                  Login.jsx               Public
```

```
/register               Register.jsx            Public
/dashboard              DashboardRedirect       Auth → role redirect
/dashboard/admin        AdminDashboard.jsx      ADMIN only 🔒
/dashboard/police       PoliceDashboard.jsx     POLICE only 🔒
/dashboard/forensic     ForensicDashboard.jsx   FORENSIC only 🔒
/dashboard/judge        JudgeDashboard.jsx      JUDGE only 🔒
```

6.3 ◆ Admin Dashboard Tab Layout

```
┌──────────────────────────────────────────────────────────────┐
│  📊 Dashboard │ 📋 Cases │ 👥 Users │ 🔍 Evidence │          │
│  🔗 Blockchain │ 🔒 Audit Logs │ 🖥 System                   │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  [Stats Cards]     [Charts/Tables]     [Action Buttons]      │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

6.4 ◆ REST API Response Format

All API responses follow this standardized format:

```
// ── Success Response ──
{
  "success": true,
  "message": "Operation description",
  "data": { ... }
}

// ── Error Response ──
{
  "success": false,
  "message": "Human-readable error description",
  "error": "TECHNICAL_ERROR_CODE"
}

// ── Paginated Response ──
{
  "success": true,
  "data": [ ... ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 150,
    "pages": 8
  }
}
```

6.5 ◆ JWT Token Payload Structure

```
{
  "id": "64f3a...",
  "userId": "64f3a...",
  "email": "officer@police.gov",
  "role": "POLICE",
  "username": "officer_raj",
  "roleId": "POL-2024-001",
  "wallet": "0xAbC1...dEf2",
  "iat": 1741651200,
  "exp": 1741737600
}
```

PART VII : SECURITY DESIGN

7.1 ◆ Authentication Security 🔒

```
Mechanism           Implementation              Detail
──────────────────────────────────────────────────────────────────
Password Hashing    bcryptjs                    Salt rounds: 10
Token Type          JWT (RS256-compatible)      24-hour expiry
Token Storage       localStorage (frontend)     Cleared on logout
Token Validation    Every protected route       Via authMiddleware.js
Admin Token         Base64 encoded special fmt  ADMIN: prefix check
Session Tracking    UserSession collection      IP + UserAgent logged
```

7.2 ◆ Role-Based Access Control (RBAC) Design 🔒

> Every API call passes through two layers of protection:

```
Layer 1 — Authentication
  ↓  authenticateToken() middleware
  ↓  Verifies JWT signature & expiry
  ↓  Attaches req.user to request

Layer 2 — Authorization
  ↓  authorizeRole(['ADMIN', 'POLICE']) middleware
  ↓  Checks req.user.role against allowed roles
```

```
  ↓  Returns 403 if role does not match

Role Capability Matrix:
                ADMIN   POLICE  FORENSIC  JUDGE
──────────────────────────────────────────────
View All Cases    ☑      ✗       ✗        ✗
Create Case       ✗      ☑       ✗        ✗
Approve Case      ☑      ✗       ✗        ✗
Assign Officers   ☑      ✗       ✗        ✗
Upload Evidence   ✗      ☑       ✗        ✗
Analyze Evidence  ✗      ✗       ☑        ✗
Submit Report     ✗      ✗       ☑        ✗
Schedule Hearing  ✗      ✗       ✗        ☑
Submit Verdict    ✗      ✗       ✗        ☑
Manage Users      ☑      ✗       ✗        ✗
View Audit Logs   ☑      ✗       ✗        ✗
Suspend Users     ☑      ✗       ✗        ✗
```

7.3 ◆ Evidence Integrity Design 🔒

> JusticeChain implements a multi-layer evidence integrity guarantee:

```
Step 1 ─► File uploaded by Police Officer
Step 2 ─► SHA-256 hash computed server-side (Node.js crypto)
Step 3 ─► File pinned to IPFS via Pinata → CID returned
Step 4 ─► IPFS CID + SHA-256 stored in MongoDB Evidence record
Step 5 ─► addEvidence(caseId, ipfsHash) called on smart contract
Step 6 ─► Contract computes keccak256(ipfsHash) → stored on chain
Step 7 ─► Any future verification:
          Recompute keccak256(ipfsHash) → compare with on-chain hash
          → Match    : Evidence is AUTHENTIC
          → Mismatch : TAMPER_DETECTED event fired
```

7.4 ◆ CORS & API Security

```
Allowed Origins   : Configured via CORS_ORIGIN env variable
                    (Vercel frontend URL in production)
Allowed Methods   : GET, POST, PUT, DELETE, OPTIONS, PATCH
Allowed Headers   : Content-Type, Authorization
Credentials       : true (supports cookies if needed)
Rate Limiting     : (Recommended: implement express-rate-limit)
Input Validation  : Mongoose schema enums + required fields
SQL Injection     : N/A (MongoDB, no raw SQL)
XSS Prevention    : JSON responses only, no raw HTML injection
```

7.5 ◆ Blockchain Security

```
Role Registration   Only admin or backend wallet can register roles
                    → Prevents unauthorized role assignment on chain

Case Operations     Require role-specific modifiers
                    → Smart contract rejects unauthorized calls

Evidence Locking    Once evidence is IMMUTABLE:
                    → Cannot be deleted or modified in MongoDB
                    → Hash permanently stored on blockchain

Verdict Finality    cases[caseId].closed = true is IRREVERSIBLE
                    → No function in contract can reopen a case
                    → Provides absolute judicial finality guarantee
```

PART VIII : DEPLOYMENT DESIGN

8.1 ◆ Production Deployment Architecture

```
                    ┌──────────────────────┐
User Browser ──────►│  Vercel CDN  [FE]    │
                    │  justice-chain.vercel│
                    │  .app                │
                    └──────────────────────┘
                              │
                              │ HTTPS
                    ┌─────────▼────────────┐
                    │  Render.com  [BE]    │
                    │  Node.js Server      │
                    │  PORT: 10000         │
                    └──────────────────────┘
                         │      │     │
                         │      │     │
            MongoDB ◄────┘      │     └──► Pinata IPFS
             Atlas              │
                                ▼
                        Ethereum Network
                        (Sepolia Testnet)
```

8.2 ◆ Environment Variables

Backend (`backend/.env`)

```
# Server
PORT=5000
NODE_ENV=production
CORS_ORIGIN=https://your-frontend.vercel.app

# Database
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/justicechain

# Auth
JWT_SECRET=your-super-secret-jwt-key

# Email
EMAIL_USER=your@gmail.com
EMAIL_PASS=your-app-password

# Blockchain
BLOCKCHAIN_PRIVATE_KEY=0xAdmin_Wallet_Private_Key
CONTRACT_ADDRESS=0xDeployed_Contract_Address
BLOCKCHAIN_RPC_URL=https://sepolia.infura.io/v3/PROJECT_ID

# IPFS / Pinata
PINATA_API_KEY=your_pinata_api_key
PINATA_SECRET_API_KEY=your_pinata_secret
```

Frontend (`src/.env`)

```
VITE_API_URL=https://your-backend.render.com
VITE_WALLETCONNECT_PROJECT_ID=your_walletconnect_id
VITE_CONTRACT_ADDRESS=0xDeployed_Contract_Address
```

8.3 ◆ Render.yaml (Backend Auto-Deploy)

```yaml
services:
  - type: web
    name: justicechain-backend
    env: node
    buildCommand: cd backend && npm install
    startCommand: cd backend && node server.js
    healthCheckPath: /health
    envVars:
      - key: NODE_ENV
        value: production
```

8.4 ◆ Deployment Checklist

```
Pre-Deploy (Backend)
  [ ] MongoDB Atlas cluster created & IP whitelist set
  [ ] All environment variables added to Render dashboard
  [ ] JusticeChain.sol deployed to Sepolia testnet
  [ ] CONTRACT_ADDRESS added to env vars
  [ ] Pinata account created, API keys configured

Pre-Deploy (Frontend)
  [ ] VITE_API_URL set to Render backend URL
  [ ] VITE_WALLETCONNECT_PROJECT_ID obtained from cloud.walletconnect.com
  [ ] VITE_CONTRACT_ADDRESS matches deployed contract

Post-Deploy Verification
  [ ] GET /health returns { status: "OK" }
  [ ] User registration + email delivery tested
  [ ] Login flow tested for all 4 roles
  [ ] Case creation → approval → assignment flow tested
  [ ] Evidence upload → IPFS pin → blockchain hash tested
  [ ] Verdict submission → contract closure tested
```

PART IX : APPENDIX

9.1 ◆ Case Lifecycle State Machine

```
                    ┌──────────┐
                    │  DRAFT   │◄── Police saves draft
                    └──────────┘
                         │
                         │ Police submits
                    ┌────▼──────────┐
                    │   REGISTERED  │
                    └───────────────┘
                         │
                         │ Police submits for approval
                    ┌────▼──────────┐
                    │ PENDING_APPROVAL │◄── Awaiting Admin
                    └───────────────┘
                         │
                         │ Admin approves
                    ┌────▼──────────┐
                    │    APPROVED   │◄── Admin assigns officers
                    └───────────────┘
                         │
                         │ Forensic officer begins analysis
                    ┌────▼──────────┐
                    │ IN_FORENSIC_  │
                    │  ANALYSIS     │
                    └───────────────┘
                         │
                         │ Forensic submits report
```

```
                    ┌──────▼──────────────┐
                    │ ANALYSIS_COMPLETE   │
                    └─────────────────────┘
                         │
                         │ Judge schedules hearing
                    ┌──────▼──────────────┐
                    │    HEARING          │
                    └─────────────────────┘
                         │
                         │ Judge submits verdict
                    ┌──────▼──────────────┐
                    │     CLOSED          │◄── Immutable on blockchain
                    └─────────────────────┘
```

9.2 ◆ Notification Design (Email Flow)

```
Event                        Recipient       Content
──────────────────────────────────────────────────────────────────────
User Registration            New User        Role ID + Welcome
Case Approval                Police Officer  Case approved notice
Forensic Assignment          Forensic Ofcr   Case assigned notice
Judge Assignment             Judge           Case assigned notice
Verdict Submitted            Police + Admin  Verdict notification
Evidence Tamper Detected     Admin           Alert notification
```

9.3 ◆ IPFS File Storage Flow

```
1. Police selects file in PoliceDashboard UI
2. Multer middleware receives multipart form
3. Server reads file buffer in memory
4. SHA-256 hash computed (crypto.createHash)
5. File uploaded to Pinata IPFS via REST API
6. Pinata returns: { IpfsHash, PinSize, Timestamp }
7. pinataUrl = "https://gateway.pinata.cloud/ipfs/{IpfsHash}"
8. Evidence document created in MongoDB with ipfsHash + pinataUrl
9. addEvidence() called on JusticeChain smart contract
10. Evidence is now decentralized + blockchain-anchored
```

9.4 ◆ Glossary

Term	Full Form / Meaning
SDD	Software Design Document
SRS	Software Requirements Specification
IPFS	InterPlanetary File System

Term	Full Form / Meaning
CID	Content Identifier (IPFS unique ID for files)
JWT	JSON Web Token
RBAC	Role-Based Access Control
CORS	Cross-Origin Resource Sharing
HMR	Hot Module Replacement (Vite feature)
EOA	Externally Owned Account (Ethereum wallet)
TX	Transaction (Blockchain transaction)
CDN	Content Delivery Network
SPA	Single Page Application
REST	Representational State Transfer
ORM	Object Relational Mapper (Mongoose for MongoDB)
ABI	Application Binary Interface (Smart Contract Interface)

9.5 ◆ Document Revision History

Version	Date	Author	Changes
1.0	March 11, 2026	Janani K — 110822104031	Initial SDD Release

---

⚖️ JusticeChain — Software Design Document Prepared by Janani K | Under the guidance of Mrs. Rajalakshmi Velammal College of Engineering and Technology

---

PART X : DETAILED SEQUENCE DESIGN

10.1 ◆ End-to-End User Registration Sequence

```
Actor: New User

1) User opens Register page (React)
2) User enters username, email, password, role
3) Frontend validates:
   - Required fields
   - Password strength
   - Confirm password match
4) Frontend sends POST /api/auth/register
```

```
5) Backend controller validates payload
6) Backend checks duplicate username/email
7) Backend hashes password using bcryptjs
8) Backend generates roleId (role-specific prefix)
9) Backend creates User document in MongoDB
10) Backend sends role credentials via emailService
11) Backend returns success response
12) Frontend shows toast + redirects to /login
```

10.2 ◆ Login & Role Routing Sequence

```
Actor: Existing User

1) User enters email + password on Login page
2) Frontend sends POST /api/auth/login
3) Backend fetches user by email
4) Backend compares password via bcrypt.compare
5) Backend verifies account status (isSuspended / isVerified)
6) Backend signs JWT with payload {id, role, roleId, wallet}
7) Backend stores user session in UserSession collection
8) Backend responds with token + user profile
9) Frontend stores session in localStorage
10) Frontend switch(role):
    ADMIN    -> /dashboard/admin
    POLICE   -> /dashboard/police
    FORENSIC -> /dashboard/forensic
    JUDGE    -> /dashboard/judge
```

10.3 ◆ Case Creation + Blockchain Anchoring Sequence

```
Actor: Police Officer

1) Police creates case in PoliceDashboard
2) FE -> POST /api/cases/create with JWT
3) authMiddleware authenticates + authorizes POLICE
4) createCase controller validates mandatory fields
5) Case document inserted in MongoDB (status REGISTERED)
6) blockchainService.createBlockchainCase() is called
7) Smart contract createCase() executed using backend signer
8) Contract emits CaseCreated(caseId, police)
9) Backend updates case with:
   - blockchainCaseId
   - blockchainCaseTxHash
   - smartContractAddress
10) API responds with DB + blockchain references
11) FE refreshes case list and shows success toast
```

10.4 ◆ Evidence Upload, IPFS Pinning & Integrity Sequence

```
Actor: Police Officer

1) Police selects file + metadata in UI
2) FE sends multipart/form-data to /api/evidence/upload
3) Multer receives file in backend memory buffer
4) Controller computes sha256Hash(fileBuffer)
5) pinataService uploads file -> receives IpfsHash CID
6) Evidence document saved with:
   - ipfsHash
   - pinataUrl
   - sha256Hash
   - status=UPLOADED
7) blockchainService.addBlockchainEvidence(caseId, ipfsHash)
8) Contract addEvidence() stores keccak256(ipfsHash)
9) Chain-of-custody entry appended (UPLOADED)
10) API returns evidence object + tx hash
11) FE updates evidence table and verification badges
```

10.5 ◆ Forensic Analysis Completion Sequence

```
Actor: Forensic Officer

1) Forensic opens assigned case
2) FE calls GET /api/evidence/:caseId
3) Forensic submits analysis report + decision
4) FE -> PUT /api/evidence/:id/analyze
5) Backend checks assignedForensic == req.user.id
6) Backend sets analysisStatus=COMPLETE
7) Optional analysis report pinned to IPFS
8) blockchainService.submitForensicReport(caseId, reportIpfsHash)
9) Contract emits ForensicSubmitted
10) Case status transitions to ANALYSIS_COMPLETE
11) Audit log entry created
```

10.6 ◆ Judicial Verdict Closure Sequence

```
Actor: Judge

1) Judge reviews forensic report and hearing notes
2) FE -> PUT /api/cases/:caseId/verdict
3) Backend validates judge assignment
4) Backend writes verdictDecision + verdictSummary
5) Backend updates case status = CLOSED
```

```
6) blockchainService.giveBlockchainVerdict(caseId, decision)
7) Contract giveVerdict() marks case.closed=true
8) Contract emits VerdictGiven
9) Backend stores blockchain tx hash
10) Notification email sent to Police + Admin
11) Case becomes read-only for final integrity
```

10.7 ◆ Admin Case Approval & Assignment Sequence

```
Actor: Admin

1) Admin opens Cases tab and reviews pending cases
2) FE -> PUT /api/cases/:id/approve
3) Backend marks case APPROVED + approvedBy
4) Backend syncs approval on blockchain approveCase()
5) Admin assigns forensic officer
6) FE -> POST /api/cases/assign-forensic
7) Backend validates forensic role and assignment state
8) Admin assigns judge
9) FE -> POST /api/cases/assign-judge
10) Backend validates judge role and case progression
11) Timeline entries recorded for each transition
```

10.8 ◆ Session Termination Sequence

```
Actor: Any Authenticated User

1) User clicks Logout
2) FE -> POST /api/auth/logout with JWT
3) Backend marks UserSession.isActive=false
4) FE clears localStorage session payload
5) FE redirects to /login
6) ProtectedRoute rejects stale navigation attempts
```

PART XI : CONTROLLER & SERVICE DESIGN

11.1 ◆ Backend Controller Responsibilities

Controller File	Primary Responsibility	Core Methods

Controller File	Primary Responsibility	Core Methods
`authController.js`	Registration, login, session and profile management	`register`, `login`, `logout`, `getMe`, `getAllUsers`
`caseController.js`	Case lifecycle operations and assignment flow	`createCase`, `approveCase`, `assignForensic`, `assignJudge`, `submitVerdict`
`evidenceController.js`	Evidence upload, integrity checks, forensic processing	`uploadEvidence`, `analyzeEvidence`, `verifyEvidence`, `lockEvidence`
`investigationController.js`	Notes, suspects, witnesses and activity logging	`addInvestigationNote`, `addSuspect`, `addWitness`, `getActivityLogs`
`reportController.js`	Forensic report retrieval and case report exports	report fetch/generation handlers
`adminRoutes.js` handlers	Admin monitoring and status collection	stats, health, user governance endpoints

11.2 ◆ Service Layer Decomposition

```
Service: blockchainService.js
Purpose: Abstract smart contract interactions from controllers
Inputs : caseId, ipfsHash, decision, role addresses
Outputs: tx hash, on-chain IDs, event confirmations
Errors : RPC_TIMEOUT, TX_REVERTED, INSUFFICIENT_GAS

Service: pinataService.js
Purpose: IPFS pin/unpin and CID lifecycle management
Inputs : file buffer, metadata, optional tags
Outputs: ipfsHash, gateway URL, pin size, timestamp
Errors : PINATA_AUTH_FAILED, PINATA_UPLOAD_FAILED

Service: emailService.js
Purpose: Email notifications for account and workflow events
Inputs : recipient, template data
Outputs: sent status / provider response
Errors : SMTP_AUTH_FAILED, EMAIL_DELIVERY_FAILED
```

11.3 ◆ Controller Method Design Template

```
Pattern used by all critical controllers:

1) Parse Request (req.body / req.params / req.user)
2) Validate Input (required fields + role checks)
3) Execute Domain Logic (business workflow)
```

```
  4) Persist Data (MongoDB write)
  5) External Side Effect (IPFS / Blockchain / Email)
  6) Record Audit Log (ActivityLog)
  7) Return JSON Response
  8) Catch Errors -> normalize via error handler
```

11.4 ◆ Error Handling Strategy

Error Category	Source	HTTP	User Message
Validation Error	Missing/invalid input	400	Invalid input. Please verify fields.
Auth Error	Missing/invalid JWT	401/403	Authentication failed. Please log in again.
Permission Error	Role mismatch	403	You are not authorized for this action.
Not Found Error	Missing case/evidence/user	404	Requested resource was not found.
Conflict Error	Duplicate assignment/status mismatch	409	Action conflicts with current state.
External Service Error	IPFS/Blockchain/Email failure	502	External service unavailable. Retry shortly.
Internal Error	Unhandled exception	500	Unexpected server error occurred.

11.5 ◆ Retry & Compensation Logic

```
Scenario A: MongoDB success, blockchain failure
  -> Mark record as PENDING_BLOCKCHAIN_SYNC
  -> Queue retry job (exponential backoff)
  -> Admin dashboard shows sync warning

Scenario B: IPFS success, MongoDB failure
  -> Store failed transaction metadata
  -> Optionally unpin orphan CID (cleanup task)

Scenario C: Email failure after critical write
  -> Keep business transaction successful
  -> Add mail retry in outbox queue
```

PART XII : FRONTEND DESIGN DETAILS

12.1 ◆ State Management Approach

JusticeChain uses a **hybrid state model**:

- **Local Component State (`useState`)** for form controls and view toggles
- **Server State (`@tanstack/react-query`)** for API data fetching and cache consistency
- **Session State (`localStorage`)** for auth token and role persistence
- **Wallet State (`wagmi`)** for connected account and network details

---

12.2 ◆ React Query Design Guidelines

```
Query Key Convention:
  ['cases', role]
  ['case', caseId]
  ['evidence', caseId]
  ['users', 'admin']
  ['audit', page]

Mutation Convention:
  useMutation for create/update operations
  onSuccess -> invalidate related query keys
  onError   -> show toast + fallback message

Cache Rules:
  staleTime: short for dashboards (near-real-time)
  refetchInterval: 30s for admin metrics
```

12.3 ◆ Routing Guard Design

```
Entry: <ProtectedRoute allowedRole="ADMIN">

Checks in order:
1) Session exists?
2) JWT token exists?
3) Role matches allowedRole?
4) Optional suspension flag?

Outcomes:
- pass  -> render dashboard page
- fail1 -> Navigate('/login')
- fail2 -> Navigate('/')
```

12.4 ◆ UI Accessibility Decisions

Area	Decision

Area	Decision
Forms	Labels and placeholders for all inputs
Buttons	Explicit action verbs (Approve, Assign, Verify)
Tables	Header labels for each column
Status	Color + text combination (not color-only semantics)
Navigation	Consistent tab ordering across sessions
Feedback	Toast + inline error text on field-level failure

12.5 ◆ Dashboard Rendering Strategy ⚡

```
Admin Dashboard
  - Initial parallel fetch for cases/users/health/stats
  - Memoized derived counters to reduce rerenders
  - Interval refresh every 30 seconds

Police Dashboard
  - Lazy-load case details when expanded
  - Paginate evidence history for large case loads

Forensic/Judge Dashboards
  - Prioritize assigned case list
  - Fetch deep detail on demand
```

12.6 ◆ Form Validation Matrix

Form	Required Fields	Validation Rules
Register	username, email, password, role	email format, password strength, role enum
Login	email, password	non-empty + valid email format
Create Case	title, description, caseNumber	minimum text lengths, unique caseNumber
Upload Evidence	caseId, title, file, type	file size/type limits, type enum
Assign Officer	caseId, officerId	officer role validation + assignment status
Verdict	caseId, decision	non-empty decision, judge assignment check

---

PART XIII : DATABASE NORMALIZATION & RULES

13.1 ◆ Data Integrity Constraints

```
users.username       -> unique, trimmed
users.roleId         -> unique, immutable after creation
cases.caseId         -> unique index
evidences.evidenceId -> unique index

Foreign-key-like references via ObjectId must resolve to existing docs:
  Case.registeredBy         -> User
  Case.assignedForensic     -> User (FORENSIC)
  Case.assignedJudge        -> User (JUDGE)
  Evidence.caseId           -> Case
  Evidence.uploadedBy       -> User
```

13.2 ◆ State Transition Rules (Case)

From State	Allowed To	Trigger
DRAFT	REGISTERED	Police submits draft
REGISTERED	PENDING_APPROVAL	Submit for admin review
PENDING_APPROVAL	APPROVED	Admin approval
APPROVED	IN_FORENSIC_ANALYSIS	Forensic starts work
IN_FORENSIC_ANALYSIS	ANALYSIS_COMPLETE	Forensic report submitted
ANALYSIS_COMPLETE	HEARING	Judge schedules hearing
HEARING	CLOSED	Judge verdict finalization

Invalid transitions must return `409 Conflict`.

13.3 ◆ State Transition Rules (Evidence)

From	To	Actor
UPLOADED	PENDING_ANALYSIS	System/Police
PENDING_ANALYSIS	ANALYZING	Forensic
ANALYZING	ANALYSIS_COMPLETE	Forensic
ANALYSIS_COMPLETE	VERIFIED	Forensic
VERIFIED	IMMUTABLE	Admin/System Lock

Once `IMMUTABLE`, update operations must be rejected except verification reads.

13.4 ◆ Data Retention Policy (Design-Level)

```
Case and evidence records      : Retained permanently
User activity logs             : Minimum 7 years
Inactive sessions              : 90-day archival
Temporary upload buffers       : Ephemeral (memory only)
Deleted/suspended user account : retained for legal traceability
```

13.5 ◆ Backup & Recovery Design

```
Primary Backup:
  MongoDB Atlas automated snapshots (daily)

Secondary Backup:
  Periodic BSON export to secure object storage

Recovery Objective:
  RPO (data loss) <= 24 hours
  RTO (service restore) <= 4 hours

Blockchain data:
  Reconstructed from chain events + stored tx hashes
```

PART XIV : NON-FUNCTIONAL DESIGN ENFORCEMENT

14.1 ◆ Performance Budget

Operation	Target Latency
Login API	< 500 ms
Case list fetch (admin)	< 1200 ms
Evidence metadata fetch	< 1000 ms
Non-file CRUD API	< 800 ms
Dashboard initial paint	< 2.5 s

14.2 ◆ Scalability Design

```
Horizontal scaling strategy:
  Backend: multiple stateless Node.js instances behind load balancer
  Database: MongoDB Atlas cluster scaling tiers
  Static FE: CDN-backed global edge delivery

Bottleneck-aware modules:
  - Evidence upload endpoints
  - Dashboard stats aggregation
  - Blockchain transaction confirmation waits
```

14.3 ◆ Availability & Resilience

Component	Failure Handling
MongoDB temporary outage	Read-only fallback messaging + retry
IPFS failure	Queue upload retry + user notification
Ethereum RPC timeout	Async sync queue + tx status polling
Email provider failure	Outbox pattern with delayed retries
Render restart	Health check + auto-recover deployment

14.4 ◆ Observability Design

```
Application Logs
  - Request ID, route, user role, status code, duration

Security Logs
  - Failed login attempts
  - Unauthorized API access (403 events)
  - Evidence tamper events

Business Logs
  - Case state transitions
  - Assignment actions
  - Verdict submissions

Infrastructure Metrics
  - CPU / memory
  - API response time percentiles
  - Error rate per endpoint
```

PART XV : DETAILED API CONTRACT APPENDIX

15.1 ◆ `POST /api/auth/register`

Request

```
{
  "username": "raj_police",
  "email": "raj@justicechain.gov",
  "password": "Strong@123",
  "role": "POLICE"
}
```

Success 201

```
{
  "success": true,
  "message": "Registration successful",
  "data": {
    "userId": "65f...",
    "roleId": "POL-2026-014"
  }
}
```

Error 409

```
{
  "success": false,
  "message": "Email already exists",
  "error": "DUPLICATE_EMAIL"
}
```

15.2 ◆ `POST /api/auth/login`

Request

```
{
  "email": "raj@justicechain.gov",
  "password": "Strong@123"
}
```

Success 200

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "token": "jwt-token-value",
    "user": {
      "id": "65f...",
      "username": "raj_police",
      "role": "POLICE",
      "roleId": "POL-2026-014",
      "wallet": "0xabc..."
    }
  }
}
```

15.3 ◆ `POST /api/cases/create`

Request

```
{
  "title": "Cyber Fraud Complaint",
  "description": "Unauthorized transfer through phishing link",
  "caseNumber": "CC-2026-00045",
  "priority": "HIGH",
  "location": "Madurai"
}
```

Success 201

```
{
  "success": true,
  "message": "Case created successfully",
  "data": {
    "caseId": "CASE-2026-0045",
    "status": "REGISTERED",
    "blockchainCaseId": 45,
    "blockchainCaseTxHash": "0x9fa..."
  }
}
```

15.4 ◆ `PUT /api/cases/:caseId/approve`

Success 200

```
{
  "success": true,
  "message": "Case approved",
  "data": {
    "status": "APPROVED",
    "blockchainApprovalTxHash": "0xf42..."
  }
}
```

15.5 ◆ `POST /api/evidence/upload`

Request: multipart/form-data

Fields:

- `caseId` (text)
- `title` (text)
- `description` (text)
- `type` (text)
- `file` (binary)

Success 201

```
{
  "success": true,
  "message": "Evidence uploaded",
  "data": {
    "evidenceId": "EVD-2026-1011",
    "ipfsHash": "Qmabc...",
    "sha256Hash": "74f81fe167d99b4cb41d6d0ccda82278...",
    "blockchainTxHash": "0x6ab..."
  }
}
```

15.6 ◆ `PUT /api/evidence/:id/analyze`

Request

```
{
  "analysisNotes": "Metadata consistent, no alteration indicators.",
  "analysisStatus": "COMPLETE",
  "analysisReport": "Forensic report summary..."
}
```

Success 200

```
{
  "success": true,
  "message": "Analysis updated",
  "data": {
    "analysisStatus": "COMPLETE",
    "status": "ANALYSIS_COMPLETE"
  }
}
```

15.7 ◆ `PUT /api/cases/:caseId/verdict`

Request

```
{
  "decision": "GUILTY",
  "summary": "Evidence corroborates prosecution claims."
}
```

Success 200

```
{
  "success": true,
  "message": "Verdict submitted",
  "data": {
    "status": "CLOSED",
    "decision": "GUILTY",
    "verdictTxHash": "0x1bc..."
  }
}
```

15.8 ◆ Common Error Payloads

```
{
  "success": false,
  "message": "Case is not in approvable state",
  "error": "INVALID_STATE_TRANSITION"
}
```

```
{
  "success": false,
  "message": "Only assigned forensic officer can analyze this evidence",
```

```
  "error": "ROLE_SCOPE_VIOLATION"
}
```

```
{
  "success": false,
  "message": "Blockchain network timeout. Please retry.",
  "error": "BLOCKCHAIN_RPC_TIMEOUT"
}
```

PART XVI : OPERATIONS & MAINTENANCE RUNBOOK

16.1 ◆ Daily Operational Checklist

```
Morning Checks
  [ ] /health endpoint returns status OK
  [ ] MongoDB connection status normal
  [ ] No failed deployments in Render logs
  [ ] API error rate < 2%

Security Checks
  [ ] No abnormal login spike
  [ ] No repeated 403/401 abuse patterns
  [ ] No tamper alerts pending acknowledgment

Blockchain Checks
  [ ] RPC provider availability stable
  [ ] Pending blockchain sync queue == 0 (or explained)
```

16.2 ◆ Incident Response Playbook

Severity	Example Incident	Response Time	Action Owner
Sev-1	Data corruption, auth bypass, major outage	< 15 min	Lead Developer + Admin
Sev-2	API degradation, delayed blockchain sync	< 1 hour	Backend Engineer
Sev-3	UI defect, minor notification failure	< 24 hours	Module Owner

---

16.3 ◆ Recovery Procedure: Blockchain Sync Failure

```
1) Identify failed records (status=PENDING_BLOCKCHAIN_SYNC)
2) Validate DB consistency for each item
3) Retry blockchain transaction with safe nonce strategy
4) Update tx hash and confirmation metadata
5) Reconcile expected state with on-chain event logs
6) Mark item SYNCHRONIZED
7) Close incident with audit note
```

16.4 ◆ Recovery Procedure: IPFS Gateway Outage

```
1) Keep accepting metadata, temporarily block file finalize step
2) Queue upload jobs locally (without losing user context)
3) Retry upload when gateway returns healthy
4) Backfill ipfsHash + pinataUrl into evidence documents
5) Execute delayed blockchain anchoring for uploaded items
```

16.5 ◆ Versioning & Release Strategy

```
Branching:
  main      -> production-ready code
  feature/* -> feature development
  hotfix/*  -> urgent production fixes

Release Tag Format:
  vMAJOR.MINOR.PATCH
  Example: v1.4.2

Change Control:
  - SRS impact check
  - SDD module update
  - Migration impact review
  - Deployment checklist execution
```

PART XVII : QUALITY ATTRIBUTES TRACEABILITY

17.1 ◆ Traceability Matrix (Quality → Design)

Quality Attribute	Design Mechanism	Component

Quality Attribute	Design Mechanism	Component
Security	JWT + RBAC + bcrypt + role guards	`authMiddleware.js`, `User.js`, `ProtectedRoute.jsx`
Integrity	SHA-256 + keccak256 + immutable ledger	`evidenceController`, `JusticeChain.sol`
Transparency	Audit logs + blockchain events	`ActivityLog.js`, contract events
Availability	Render health checks + MongoDB Atlas SLA	`app.js`, deployment config
Maintainability	Layered module structure	routes/controllers/services split
Scalability	Stateless API + horizontal backend scaling	Express + CDN architecture
Traceability	case timeline + chainOfCustody	`Case.js`, `Evidence.js`

17.2 ◆ Risk Register (Design-Centric)

Risk ID	Risk Description	Impact	Mitigation
R-01	Blockchain gas spikes delay writes	Medium	Async queue + retry + status badges
R-02	IPFS downtime blocks upload completion	High	Deferred queue + reprocess worker
R-03	Role misconfiguration in user data	High	strict enum + admin validation + audit
R-04	Large evidence payload memory pressure	Medium	file-size limit + streaming strategy roadmap
R-05	Token leakage from client storage	High	shorter expiry + rotation + HTTPS-only transport
R-06	Partial DB-chain inconsistency	High	compensation logic + reconciliation jobs

17.3 ◆ Technical Debt Backlog (Planned Enhancements)

```
TD-01  Introduce centralized error middleware
TD-02  Add API rate limiter and brute-force protection
TD-03  Add pagination for all heavy list endpoints
TD-04  Introduce background job queue (BullMQ/RabbitMQ)
TD-05  Add OpenAPI/Swagger contract generation
TD-06  Implement end-to-end integration test suite
TD-07  Add SIEM-compatible structured security logging
TD-08  Add wallet-signature challenge on login
```

17.4 ◆ Document Revision History (Extended Edition)

Version	Date	Author	Changes
1.0	March 11, 2026	Janani K — 110822104031	Initial SDD Release
1.1	March 11, 2026	Janani K — 110822104031	Extended SDD edition expanded with detailed sequence design, service internals, API contract appendix, operations runbook, and quality traceability for up-to-30-page content target

---

☑ EXTENDED SDD COMPLETE (UP TO 30-PAGE CONTENT EDITION)

JusticeChain — Detailed Design Reference